Studio Nx

Backend(FastAPI/Python) 29 — authenticate() · invalid_credentials()

2026-10-07

Backend(FastAPI/Python) 29 — `authenticate()` · `invalid_credentials()`

항목	내용
파일	`studionx/backend/app/services/auth_service.py`
작업	P1-05 로그인 — 이메일 · 비밀번호 확인 · PRD 01 FR-103 · FR-108
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	28 `decode_refresh_token()` · 30 `get_active_user()`

1. 하는 일

로그인 모달에 입력한 이메일 · 비밀번호가 맞는지 확인한다. 맞으면 회원을 돌려주고 **마지막 로그인 시각**을 기록한다.

경우	결과	로그인 모달 문구
이메일 · 비밀번호 맞음, 활동 회원	`User` (+ `last_login_at` = 지금)	— (로그인됨)
비밀번호 틀림	401 `INVALID_CREDENTIALS`	Email or password is incorrect.
없는 이메일	401 `INVALID_CREDENTIALS` — 위와 **같은 문구 · 같은 시간**	Email or password is incorrect.
탈퇴 회원 (비밀번호 맞아도)	401 `INVALID_CREDENTIALS` — 위와 같음	Email or password is incorrect.
정지 회원, 비밀번호 **맞음**	403 `ACCOUNT_DISABLED`	This account is disabled.
정지 회원, 비밀번호 틀림	401 `INVALID_CREDENTIALS`	Email or password is incorrect.

함수	역할
`authenticate()`	확인 · 기록
`invalid_credentials()`	401 오류를 만듦 (네 군데에서 같은 오류를 씀)

2. 언제 쓰나

쓰는 곳	화면
P1-05 `POST /auth/login` (32)	home.html 로그인 모달의 **Sign In** 버튼

32 가 이 함수로 회원을 확인한 뒤, 26 · 27 로 토큰 두 개를 만들어 준다.

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
```

```
from app.core.security import check_password_rules, hash_password, verify_password
from app.models import User
from app.models.user import UserStatus
from app.schemas.auth import PRIVACY_VERSION, AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import 이다. **`verify_password` 를 이번에 추가했다.** 이 함수는 `datetime` · `UTC` · `select` · `Session` · `AppError` · `ErrorCode` · `verify_password` · `User` · `UserStatus` 를 쓴다.

함수

```
# 없는 이메일로 로그인할 때 비교에 쓰는 가짜 해시 (bcrypt 비용 12 — 진짜 해시와 같은 시간이 걸림).
# 어떤 비밀번호와도 맞지 않는다. 비밀값이 아니므로 코드에 두어도 됨.
DUMMY_PASSWORD_HASH = "$2b$12$1YCmgxF3pcmPkm7MWpsgVuOD.w6LlO7o43SuOZLbLZkz.glHDfmva"


def authenticate(db: Session, email: str, password: str) -> User:
    user = db.scalar(select(User).where(User.email == normalize_email(email)))
    if user is None:
        verify_password(password, DUMMY_PASSWORD_HASH)  # 없는 이메일도 같은 시간(약 0.2초)이 걸리게
        raise invalid_credentials()
    if not verify_password(password, user.password_hash):
        raise invalid_credentials()
    if user.status == UserStatus.DELETED:
        raise invalid_credentials()
    if user.status == UserStatus.SUSPENDED:
        raise AppError(ErrorCode.ACCOUNT_DISABLED, 403, "This account is disabled.")
    user.last_login_at = datetime.now(UTC).replace(tzinfo=None)
    db.commit()
    return user


def invalid_credentials() -> AppError:
    return AppError(ErrorCode.INVALID_CREDENTIALS, 401, "Email or password is incorrect.")
```

문법 풀이

import 문법	뜻
`from app.core.security import …, verify_password`	16 의 비교 함수를 가져옴

식	뜻
`verify_password(password, DUMMY_PASSWORD_HASH)`	결과를 **쓰지 않고** 부르기만 함 — 시간만 쓰게 하려는 것
`raise invalid_credentials()`	함수가 만든 오류를 던짐 (21 의 `unavailable_error` 와 같은 방식)
`user.last_login_at = …` → `db.commit()`	회원 행의 마지막 로그인 시각을 바꿔 저장
`.replace(tzinfo=None)`	DB 칸(`DATETIME`)은 시간대 표시가 없어서 UTC 시각에서 표시를 뗌 (19 와 같음)

4. 흐름

1. 이메일을 17 로 맞춰 회원을 찾는다.
2. 없으면 → 가짜 해시로 한 번 비교(시간 맞추기) → 401.

3. 비밀번호가 틀리면 → 401.
4. 탈퇴 회원이면 → 401 (같은 문구).
5. 정지 회원이면 → 403.
6. 마지막 로그인 시각을 기록하고 회원을 돌려준다.

5. 왜 이렇게

선택	이유
틀린 이유를 알려 주지 않음	"이메일이 없음" 과 "비밀번호 틀림" 을 구분해 주면, 남의 이메일이 가입됐는지 알아낼 수 있다 (FR-103 통일 문구)
없는 이메일도 가짜 해시로 비교	비교(약 0.2초)를 건너뛰면 없는 이메일만 빨리 답한다. 응답 **시간**으로 가입 여부가 드러나는 것을 막는다 (16 §5 에 적어 둔 일)
가짜 해시를 코드에 상수로	서버가 켜질 때마다 0.2초씩 만들 필요가 없다. 비밀번호가 아니라 "아무것과도 맞지 않는 해시" 라 드러나도 문제없다. 비용 12 라 진짜 해시와 시간이 같다
탈퇴 회원도 같은 401	탈퇴한 계정이 있었다는 사실도 알려 주지 않는다 (18 의 추적 방지와 같은 생각)
정지(403)는 **비밀번호가 맞은 뒤에만** — 템플릿 순서 그대로	틀린 비밀번호로는 "이 이메일은 정지됐다" 를 알 수 없다. 본인(비밀번호를 아는 사람)에게만 정지 사실을 알려 준다
실패하면 `last_login_at` 을 바꾸지 않음	성공한 로그인만 기록한다

참고 — 탈퇴 3일 뒤: 18 로 비켜 둔 행은 이메일이 `deleted+…` 로 바뀌어 원래 이메일로는 찾을 수조차 없다. P1-09 정리 작업은 비밀번호 해시를 `"!"` 로 바꾸므로, 16 이 늘 `False` 를 준다.

6. 시험 (`tests/test_auth_service.py` 에 7개 추가 · 모두 44개, 실제 MySQL)

시험	확인한 것
성공	그 회원을 돌려주고 `last_login_at` 이 지금(5초 안)으로 기록됨
대소문자 · 공백	`" LOGIN@Example.com "` 로도 로그인됨 (17)
비밀번호 틀림	401 "Email or password is incorrect.", `last_login_at` 그대로
없는 이메일	같은 401 · 같은 문구, 걸린 시간 0.1초 이상 (가짜 해시 비교를 했음)
탈퇴 회원 · 맞는 비밀번호	401 (틀린 비밀번호와 구분 안 됨)
정지 회원	맞는 비밀번호 → 403 "This account is disabled.", 틀린 비밀번호 → 401
가짜 해시	`""` · `"password"` · `"Abcdef1!"` 등 어떤 것과도 맞지 않음

시험 도우미 `add_user()` 가 `password_hash` 를 바꿔 넣을 수 있게 고쳤다 (진짜 해시로 로그인 시험).

전체 시험 209개 통과, `ruff` 검사 통과. 시험 뒤 남은 행 0개.